

```

.panel{background:var(--panel);padding:12px;border-radius:10px;box-shadow:0 6px 18px rgba(0,0,0,0.06)}

.segmented{display:flex;gap:6px;margin-bottom:10px}

.seg-btn{flex:1;padding:8px;border-radius:8px;border:1px solid rgba(0,0,0,0.06);background:#fff;cursor:pointer}

.seg-btn.active{background:linear-gradient(90deg,var(--purple),#4b0000);color:#fff}

.order-container{display:none;gap:8px;flex-direction:column}

.order-container.active{display:flex}

label{display:flex;flex-direction:column;margin:6px 0;font-size:0.9rem}

input,select{padding:8px;border-radius:6px;border:1px solid #ddd}

.calc{margin:8px 0;color:var(--muted)}

.cards{display:flex;flex-direction:column;gap:10px}

.card{background:var(--card);padding:10px;border-radius:8px;border:1px solid rgba(0,0,0,0.04)}

.muted.small{color:var(--muted);font-size:0.85rem}

.statusbar{padding:8px 16px;color:var(--muted);font-size:0.9rem}

@media (min-width:900px){

    .order-container{flex-direction:row;align-items:center}

    .order-container label{flex:1;margin:0 8px}

    .segmented{max-width:480px}

}

```

app.js

```
/* app.js
```

```
PortalX LITE Test Project front-end
```

```
- Hook points / comments explain where to wire StellarWalletsKit and Horizon calls.
```

```
- Place these files in your repo root (index.html, style.css, app.js).
```

- Put the three test JSON files into /data/ (path /data/Test BaseQuote.json etc).

*/

```
const STATUS = document.getElementById('status');

const connectBtn = document.getElementById('connectBtn');

const importBtn = document.getElementById('importBtn');

const importFile = document.getElementById('importFile');

const updateBtn = document.getElementById('updateBtn');

const exportJsonBtn = document.getElementById('exportJsonBtn');

const exportXlsxBtn = document.getElementById('exportXlsxBtn');


const offersContainer = document.getElementById('offersContainer');

const offersMsg = document.getElementById('offersMsg');


const buyMarket = document.getElementById('buyMarket');

const sellMarket = document.getElementById('sellMarket');

const swapFrom = document.getElementById('swapFrom');

const swapTo = document.getElementById('swapTo');


const buyPrice = document.getElementById('buyPrice');

const buyBaseTotal = document.getElementById('buyBaseTotal');

const buyEstimatedQuote = document.getElementById('buyEstimatedQuote');


const sellPrice = document.getElementById('sellPrice');

const sellQuoteAmount = document.getElementById('sellQuoteAmount');

const sellEstimatedBase = document.getElementById('sellEstimatedBase');
```

```

const swapAmount = document.getElementById('swapAmount');
const swapSlippage = document.getElementById('swapSlippage');
const swapEstimate = document.getElementById('swapEstimate');

const placeBuyBtn = document.getElementById('placeBuyBtn');
const placeSellBtn = document.getElementById('placeSellBtn');
const placeSwapBtn = document.getElementById('placeSwapBtn');

const logContainer = document.getElementById('logContainer');
const logEmpty = document.getElementById('logEmpty');

let connectedAddress = null;
let baseQuoteList = []; // loaded from Test BaseQuote.json
let assetsList = []; // loaded from Test Universal Asset List.json
let orderLog = []; // loaded from Test Order Log.json or created locally

// ----- UI: segmented controls -----
document.querySelectorAll('.seg-btn').forEach(b=>{
  b.addEventListener('click', ()=>{
    document.querySelectorAll('.seg-btn').forEach(x=>x.classList.remove('active'));
    b.classList.add('active');
    const t = b.dataset.target;
    document.querySelectorAll('.order-
container').forEach(c=>c.classList.remove('active'));
    document.getElementById(t).classList.add('active');
  });
});

```

```

});

});

// ----- status helper -----

function setStatus(s){
    STATUS.textContent = 'Status: ' + s;
}

// ----- load test JSON files from /data -----

async function loadTestFiles(){
    setStatus('Loading test JSON files...');
    try{
        // adjust paths if needed

        const baseQ = await fetch('/data/Test BaseQuote.json').then(r=>r.json());
        const universal = await fetch('/data/Test Universal Asset
List.json').then(r=>r.json());
        const log = await fetch('/data/Test Order
Log.json').then(r=>r.json()).catch(()=> null);

        // baseQ is luckysheet format (array with one sheet) – we will extract rows
with celldata

        baseQuoteList = extractSimpleRowsFromLuckysheet(baseQ);
        assetsList = extractSimpleRowsFromLuckysheet(universal);
        if(log && Array.isArray(log)) {
            orderLog = extractSimpleRowsFromLuckysheetToObjects(log);
        } else if (Array.isArray(log)) {
            orderLog = log;

```

```

    } else {
        orderLog = [];
    }

    populateMarkets();
    populateAssets();
    renderLog();
    setStatus('Test JSON loaded');
} catch(err){
    console.error(err);
    setStatus('Could not load /data files – check console and file paths');
}
}

// Convert the luckysheet JSON exported structure to simple rows (assumes first
sheet)

// returns array of rows (arrays of cell values)
function extractSimpleRowsFromLuckysheet(sheetJson){
    if(!Array.isArray(sheetJson) || sheetJson.length===0) return [];
    const s = sheetJson[0];
    const out = [];
    const cells = s.celldata || [];
    // find max r
    let maxr = 0;
    cells.forEach(c=>{ if(c.r>maxr) maxr=c.r; });
    for(let r=0;r<=maxr;r++){

```

```

    out[r] = [];
  }
  // fill values
  cells.forEach(c=>{
    const v = (c.v && c.v.v !== undefined) ? c.v.v : (c.v && c.v.m) ? c.v.m : '';
    out[c.r][c.c] = v;
  });
  // trim empty rows
  return out.map(row => row.map(cell => cell===undefined?'':cell));
}

// Convert order log luckysheet to objects using header row
function extractSimpleRowsFromLuckysheetToObjects(sheetJson){
  const rows = extractSimpleRowsFromLuckysheet(sheetJson);
  if(rows.length<1) return [];
  const headers = rows[0].map(h=>String(h).trim());
  const objs = [];
  for(let i=1;i<rows.length;i++){
    const r = rows[i];
    if(r.join('').trim()=== '') continue;
    const o={};
    headers.forEach((h,ci)=> o[h] = r[ci]!==undefined? r[ci] : '');
    objs.push(o);
  }
  return objs;
}

```

```

function populateMarkets(){

    // baseQuoteList rows expected: header row at [0]. Market: col 0, Base asset
    col1, Quote col2

    const rows = baseQuoteList.slice(1).filter(r=>r[0]);

    buyMarket.innerHTML = '';

    sellMarket.innerHTML = '';

    rows.forEach(r=>{

        const opt = document.createElement('option'); opt.value = r[0]; opt.textContent
        = r[0];

        buyMarket.appendChild(opt.cloneNode(true));

        sellMarket.appendChild(opt.cloneNode(true));

    });
}

```

```

function populateAssets(){

    // assetsList rows expected: header at row 0; asset code in col 0

    const rows = assetsList.slice(1).filter(r=>r[0]);

    swapFrom.innerHTML = ''; swapTo.innerHTML = '';

    rows.forEach(r=>{

        const code = r[0];

        const opt = document.createElement('option'); opt.value = code; opt.textContent
        = code;

        swapFrom.appendChild(opt.cloneNode(true));

        swapTo.appendChild(opt.cloneNode(true));

    });
}

```

```

// ----- log rendering -----
function renderLog(){
  logContainer.innerHTML = '';
  if(!orderLog || orderLog.length===0){
    logEmpty.style.display = 'block';
    return;
  }
  logEmpty.style.display = 'none';
  orderLog.slice().reverse().forEach((row, idx)=>{
    const card = document.createElement('div'); card.className='card';
    card.innerHTML = `
      <div><strong>${row.market || row.Market || row.market || '-'}</strong> – $
      {row.type || ''}</div>
      <div class="muted small">Last updated: ${row['Last Updated Date, Time'] ||
      row.timestamp || ''}</div>
      <div style="margin-top:6px">offer_id: ${row.offer_id||row['Stellar
      offer_id']||''}</div>
    `;
    logContainer.appendChild(card);
  });
}

// ----- import local JSON file -----
importBtn.addEventListener('click', ()=> importFile.click());
importFile.addEventListener('change', async (e)=>{
  const f = e.target.files[0];

```



```

if(!f) return;

setStatus('Importing ' + f.name);

try{

  const txt = await f.text();

  const data = JSON.parse(txt);

  // simple heuristic: if array -> assume luckysheet format -> try to extract to
objects

  if(Array.isArray(data) && data[0] && data[0].celldata){

    const objs = extractSimpleRowsFromLuckysheetToObjects(data);

    orderLog = orderLog.concat(objs);

  } else if(Array.isArray(data)){

    // assume it's the order log array

    orderLog = orderLog.concat(data);

  } else if(data && data.celldata){

    orderLog = orderLog.concat(extractSimpleRowsFromLuckysheetToObjects([data]));

  } else {

    // fallback: if object with header keys

    orderLog.push(data);

  }

  renderLog();

  setStatus('Imported ' + f.name);

}catch(err){

  console.error(err);

  setStatus('Import failed – check console');

} finally {

  importFile.value='';

```

```

    }
  });

// ----- Update Offers (Active Offers) -----
updateBtn.addEventListener('click', async ()=>{
  setStatus('Fetching active offers (Horizon)...');
  offersContainer.innerHTML = '';
  offersMsg.style.display='none';
  try{
    // NOTE: Direct Horizon calls may require CORS and may be rate-limited.
    // For testing you can host a small JSON at /data/activeOffers.json and this
    // will read it.
    // Replace with your Horizon endpoint fetch when you have a proxy or server
    // setup.
    const resp = await
    fetch('/data/activeOffers.json').then(r=>r.json()).catch(()=>null);

    if(!resp){
      offersMsg.style.display='block';

      offersMsg.textContent = 'No test activeOffers.json found in /data. Put sample
      offers there when testing.';

      setStatus('No offers (using /data fallback)');

      return;
    }

    // render sample
    resp.records?.forEach(o=>{
      const card = document.createElement('div'); card.className='card';

      card.innerHTML = `<div><strong>${o.selling?.asset_code||'XLM'} / $
      {o.buying?.asset_code||'XLM'}</strong> – offer_id ${o.id}</div>

```

```

        <div class="muted small">price: ${o.price} – amount: ${o.amount}</div>`;
    offersContainer.appendChild(card);
});

setStatus('Offers updated');
}catch(err){
    console.error(err);
    setStatus('Error fetching offers – see console');
}
});

// ----- Connect wallet (placeholder wiring for StellarWalletsKit) -----
connectBtn.addEventListener('click', async ()=>{
    setStatus('Connecting wallet...');
    try{
        // If you have StellarWalletsKit already loaded in your page (from your
        bundle), use it:
        if(window.StellarWalletsKit){
            try{
                // Example usage (your environment may vary)
                const auth = await StellarWalletsKit.authModal();
                if(auth && auth.address){
                    connectedAddress = auth.address;
                    connectBtn.textContent = 'Connected: ' + shortenAddress(auth.address);
                    setStatus('Wallet connected: ' + auth.address);
                } else {
                    setStatus('Wallet auth returned no address');
                }
            }
        }
    }
});

```

```

    }
  }catch(e){
    console.error('StellarWalletsKit auth failed', e);
    setStatus('Wallet connect failed (StellarWalletsKit)');
  }
} else {
  // Fallback: show an instruction modal to explain how to wire
  StellarWalletsKit

  alert('No StellarWalletsKit found. To enable WalletConnect, deploy using your
  existing portalx-lite test setup that includes StellarWalletsKit (we have working
  example in your portalx-lite-test repo).');

  setStatus('StellarWalletsKit not present');
}
}catch(err){
  console.error(err);
  setStatus('Connect error – check console');
}
});

function shortenAddress(a){
  if(!a) return '';
  return a.slice(0,6) + '...' + a.slice(-6);
}

// ----- Auto-calculation helpers -----
function toNum(v){ return (v===undefined||v==='')?0:Number(v); }

```

```

function updateBuyEstimate(){
    const price = toNum(buyPrice.value);
    const total = toNum(buyBaseTotal.value);
    if(!price || !total) { buyEstimatedQuote.textContent='-'; return; }
    // price is Quote in Base => Quote = total / price
    const q = total / price;
    buyEstimatedQuote.textContent = q.toFixed(7);
}

function updateSellEstimate(){
    const price = toNum(sellPrice.value);
    const amount = toNum(sellQuoteAmount.value);
    if(!price || !amount){ sellEstimatedBase.textContent='-'; return; }
    // base received = amount / price
    const b = amount / price;
    sellEstimatedBase.textContent = b.toFixed(7);
}

buyPrice.addEventListener('input', updateBuyEstimate);
buyBaseTotal.addEventListener('input', updateBuyEstimate);
sellPrice.addEventListener('input', updateSellEstimate);
sellQuoteAmount.addEventListener('input', updateSellEstimate);

// ----- Place order handlers (simulate payloads + instructions) -----
placeBuyBtn.addEventListener('click', async ()=>{

    // For PortalX LITE convention: Buy order is fixed Base Total (we must create a
    manageSellOffer with selling=base)

    const market = buyMarket.value;

```

```

const price = buyPrice.value;

const baseTotal = buyBaseTotal.value;

if(!market || !price || !baseTotal) { alert('Complete all fields'); return; }

const [quote, base] = market.split('/'); // market format is "QUOTE/BASE" in Test
BaseQuote (you used "AFR/USDC" etc)

// IMPORTANT: confirm your market encoding. Here Test BaseQuote used "Market
(Quote/Base)" as "AFR/USDC" meaning Quote=AFR,Base=USDC

const payload = {

  action: 'create-buy-via-sell-offer',

  market, price, baseTotal,

  // For building manageOffer on Stellar you will need which asset is selling and
  which buying

  // For PortalX LITE: since baseTotal is fixed, we treat base as the SELLING
  asset for manageSellOffer

  selling: base,

  buying: quote,

  selling_amount: Number(baseTotal).toFixed(7)

};

// Show preview to user:

showOrderPreview('Buy (simulated)', payload);

// TODO: Build stellar transaction using Stellar SDK, sign with
StellarWalletsKit, submit to Horizon.

// Example pseudo:

// 1) Build Transaction with ManageSellOffer operation (selling = base asset,
buying = quote, amount = baseTotal, price = price)

```

```

    // 2) Get XDR, call StellarWalletsKit.signTransaction(xdr, {networkPassphrase,
address})

    // 3) Submit signed tx to Horizon via fetch / server proxy.

});

placeSellBtn.addEventListener('click', async ()=>{

    const market = sellMarket.value;

    const price = sellPrice.value;

    const quoteAmount = sellQuoteAmount.value;

    if(!market || !price || !quoteAmount) { alert('Complete all fields'); return; }

    const [quote, base] = market.split('/');

    const payload = {

        action: 'create-sell-offer',

        market, price, quoteAmount,

        // For selling quote asset (user specified quote amount is fixed)

        selling: quote,

        buying: base,

        selling_amount: Number(quoteAmount).toFixed(7)

    };

    showOrderPreview('Sell (simulated)', payload);

    // TODO: Build ManageSellOffer or ManageBuyOffer depending on convention.

    // Example: Use ManageSellOffer(selling=quote, buying=base, amount=quoteAmount,
price=price)

});

```

```

placeSwapBtn.addEventListener('click', async ()=>{

  const from = swapFrom.value;

  const to = swapTo.value;

  const amount = swapAmount.value;

  if(!from || !to || !amount){ alert('Complete swap fields'); return; }

  setStatus('Estimating path (Horizon) ...');


  // Use Horizon pathfinding endpoint (strict-send or strict-receive) – requires
  CORS / proxy.

  // For example (pseudo):

  // POST /paths/strict-send with source_asset, source_amount, destination_asset

  // The response includes path and estimated destination amount.

  // Below we simulate an estimated result; in production replace with a real fetch
  to Horizon / Lobstr endpoint.


  // Simulated estimate:

  const simulatedReceive = Number(amount) * 0.98; // dummy slippage

  swapEstimate.textContent = simulatedReceive.toFixed(7);

  setStatus('Swap estimate ready (simulated). Replace with Horizon path call for
  real estimate.');
```

```

const payload = {

  action: 'swap-simulated',

  from, to, amount, estimated_receive: simulatedReceive

};

showOrderPreview('Swap (simulated)', payload);
```



```

    // TODO: Build path payment or strict-send op, sign via StellarWalletsKit, submit
    to Horizon

});

// Preview + add to local log

function showOrderPreview(title, payload){

    // Simple modal via confirm / prompt replacement; for now append to log and
    render

    const keep = confirm(`${title}\n\nPayload preview:\n$
{JSON.stringify(payload,null,2)}\n\nClick OK to add to local Order Log
(simulated).`);

    if(keep){

        const now = new Date().toLocaleString('en-US', {timeZone:'America/New_York'});

        const entry = {

            'Last Updated Date, Time': now,

            market: payload.market || `${payload.selling}/${payload.buying}`,

            type: payload.action,

            buy_total_base: payload.baseTotal || '',

            price_quote_in_base_start: payload.price || '',

            sell_amount_quote: payload.selling_amount || '',

            offer_id: '' // filled after real Horizon submission

        };

        orderLog.push(entry);

        renderLog();

        setStatus('Order added to local log (simulated)');

    } else {

        setStatus('Order preview dismissed');
    }
}

```

```

    }
}

// ----- export functions -----

exportJsonBtn.addEventListener('click', ()=>{
    const blob = new Blob([JSON.stringify(orderLog, null, 2)],
    {type:'application/json'});

    const url = URL.createObjectURL(blob);

    const a = document.createElement('a'); a.href = url; a.download =
    'PortalX_TestOrderLog.json'; a.click();

    URL.revokeObjectURL(url);
});

exportXlsxBtn.addEventListener('click', ()=>{
    // simple conversion: array of objects -> sheet

    if(!orderLog || orderLog.length===0) { alert('No log to export'); return; }

    const ws = XLSX.utils.json_to_sheet(orderLog);

    const wb = XLSX.utils.book_new();

    XLSX.utils.book_append_sheet(wb, ws, 'OrderLog');

    XLSX.writeFile(wb, 'PortalX_TestOrderLog.xlsx');
});

// ----- init -----

(async function init(){
    setStatus('Initializing UI...');

    await loadTestFiles();

    setStatus('Ready');

```

```
})();
```

Next steps & guidance (practical)

1. Paste the three files above into your `portalx-lite-wc-test` project folder (or whatever test repo you created).
 - Put your `Test BaseQuote.json`, `Test Order Log.json`, `Test Universal Asset List.json` inside the `/data` folder in that repo (same structure we used when testing).
 - Put `wallet-icon.png` in `/assets` (or update `<img>` path in `index.html`).
2. Open in **Vite** or serve statically (GitHub Pages):
 - If using GitHub Pages: commit & push, then `npm run build + npm run deploy` as you did before for `portalx-lite-test` (or simply push and let GitHub Pages serve the static files if you're not using build tooling).
 - For local testing where you used `StellarWalletsKit` previously, run your normal local dev flow (Vite) since you already have that working in the other repo.
3. Hook `WalletConnect` / `StellarWalletsKit`:
 - In `app.js` connect the `connectBtn` handler portion with `StellarWalletsKit.authModal()` (you already have working implementation in the other repo — copy that exact pattern).
 - For placing real orders: you will need to construct Stellar Transaction (using Stellar SDK) and call `StellarWalletsKit.signTransaction(xdr, { networkPassphrase, address })`, then submit signed XDR to a Horizon endpoint. I left `TODO` comments where to implement that.
4. Horizon CORS / submission:
 - Fetching live path estimates or posting transactions often requires a server proxy (unless Lobstr/Horizon endpoint allows cross-origin requests from GH Pages). If you hit CORS headaches, use a small server (or lambda) to proxy requests.
5. When you're ready I can:

- Inject the exact `StellarWalletsKit` call code you used successfully in the `portalx-lite-test` repo (so this UI will call the same wallet kit and the signing flow will match).
 - Add the exact `manageBuyOffer/manageSellOffer` building code using Stellar SDK (example code + how to include the SDK in your build).
 - Create an example `activeOffers.json` to place in `/data` so Update Offers renders live on first deploy.
-

If you want I can also:

- Produce a **single commit-ready zip** of these files arranged in the repo structure (`index.html`, `style.css`, `app.js`, `/data` placeholders) — but I can't upload files for you (you'll paste).
- Or immediately inject your working `StellarWalletsKit` snippets into `app.js` if you paste the exact working code you used in your `portalx-lite-test` repo for authentication & signing (I'll merge it for you).

Shall I:

- (A) Add the actual `StellarWalletsKit` auth & sign calls into this `app.js` now (if you paste your working snippet from `portalx-lite-test`), **or**
- (B) Leave `app.js` as-is and next we wire the live signing/submission & Horizon calls when you're ready?

I'll wait for your pick.

Ok Poof that was a lot to unpack, so I decided to leave it until morning, but I did wake up in the middle of the night and copy/paste the code you gave me to their respective files, but there were other things in your plan that confused me like what to do after adding the code. Like I need specific instructions on what I should use to commit and push to GitHub. I still don't have that down. I know you've described how commits should be documented, but it still seems foreign to me. Like you used: chore, feat, fix, and docs and gave suggested messages for each, but I still don't know all of what was done or how to generalize it or categorize it for commit messages at this stage. Also, you mentioned Vite and npm again, but I thought we weren't using those and would be "CDN" to build everything. The "commit-ready zip" seemed a little advanced, so I didn't really understand what that would entail. I think I'd rather just do commits the way we were doing them since I haven't even mastered that yet. As far as the auth & sign call in `portalx-lite-test`, I don't see that we included the sign transaction code